Multivariate Legendre-SNN on Loihi-2 for Time Series Classification and 5G Jamming Detection

Ramashish Gaurav, Sujata Sinha, Chunxiao Lin, Terrence C. Stewart, Lingjia Liu, and Yang Yi, *Senior Member, IEEE*

***Abstract*—5G-&-Beyond technologies offer the promise of improved speed and bandwidth, ultra low latency, high network reliability, and have the potential to enable new applications and services. It only seems fitting to complement the transformative future of 5G-&-Beyond with the *low energy* offering of Spiking Neural Networks (SNNs) on neuromorphic chips. In this work, we develop Loihi-2 (Intel's neuromorphic chip) -compatible versions of our previously proposed Legendre-SNN model for univariate and multivariate Time-Series Classification (TSC), as well as for 5G wireless applications. The Legendre-SNN is a reservoir-based SNN, where, the *non-spiking* Legendre Delay Network (LDN) is used as a static reservoir, followed by a trainable *spiking* network. Deploying such an SNN model (mix of *non-spiking* and *spiking* components) *entirely* on Loihi-2 is nontrivial - this is due to the scarcity of related approaches and technical documentations. In this work, we present our approach and the technicalities of implementing the *non-spiking* LDN on the rarely used "Lakemont core" (embedded on Loihi-2); thereby, adding to the scarce technical documentation to program on-chip Lakemont cores. Thus, our presented approach can be leveraged by other researchers as well - to implement their *non-spiking* components right on-chip. Our proposed *hardware-friendly versions* of Legendre-SNN when evaluated on *Loihi-2*, outperform LSTM-based models (- executed on *GPU*) on 7 of 24 TSC datasets. Here, we also emphasize on the applications of our Legendre-SNN versions for 5G Jamming Detection on Loihi-2, and complement it with a real-time video demonstration of Jamming Detection (with simulated signals) on our physical Kapoho-Point Single Chip Loihi-2 board, followed by detailed energy-analysis. Overall, this work is directed towards the (comparatively) understudied technical side of neuromorphic computing to enable researchers leverage the Lakemont cores and deploy their SNNs *entirely* on Loihi-2, with a push towards the cause for neuromorphics in Wireless Communications.**

***Index Terms*—Reservoir computing, spiking neural networks, neuromorphic hardware, Loihi-2, wireless, jamming detection, time series classification, legendre delay network, LSTM.**

I. Introduction

EDGE Computing (EC) / Internet of Things (IoT) are becoming increasingly ubiquitous, fueled by further advancements in Wireless Communications and Edge AI. Simultaneously, for portability reasons, EC/IoT devices are leaning towards battery powered systems, thereby, also harnessing the battery technology advancements. It is only in the interest of prolonging the battery life and for Green AI [1], [2], [3] that we should strive to develop energy-efficient Edge AI systems. Neuromorphic Computing with Spiking Neural Networks (SNNs) is gradually emerging to be a promising potential candidate for energy-efficient AI. Apart from the conventional AI tasks, the SNNs have demonstrated their potential across a variety of EC/IoT [4], [5], [6] and Wireless Communication [7], [8], [9] tasks, although, in most of the works, the proposed SNN architectures are *not actually* deployed and evaluated on a neuromorphic hardware; this undermines the practicality of SNNs for real-world applications. A few reasons for the lack of works on real-world SNN deployment on a neuromorphic hardware are: (1) the associated technical difficulty and (2) the limited technical documentation. Therefore, in this work, we take the example of Intel's latest neuromorphic research chip: Loihi-2, and shed light on the technical specifics of deploying and evaluating our previously proposed Legendre-SNN [10] on the Kapoho Point Single Chip (KPSC) Loihi-2 neuromorphic board (from Intel); which we also use for a wireless application.

Legendre-SNN [10] is a novel reservoir-based SNN where the Legendre Delay Network (LDN) [11], [12] is used as a *static* reservoir to extract temporal features (from the *univariate* input signal) that are encoded to spikes, over which, an SNN following the reservoir learns non-linear mapping to the output classes; we present more details of Legendre-SNN in the Sec II-C. A peculiar aspect of the Legendre-SNN is that the *non-trainable* reservoir constitutes only of regular *linear matrix operations*, i.e., *no* neurons of any kind are involved. Thus, the Legendre-SNN comprises a *non-spiking* reservoir followed by a *spiking* network (composed of an encoding layer, one spiking hidden layer, and an output layer); implementing such an SNN model *entirely* on Loihi-2 is non-trivial because the existing

Received 4 July 2025; revised 27 November 2025; accepted 13 January 2026. Date of publication 28 January 2026; date of current version 31 July 2026. This work was supported by U.S. National Science Foundation (NSF) under Grant CCF-1750450, Grant ECCS-1731928, Grant ECCS-2128594, Grant ECCS- 2314813, and Grant CCF-1937487. The review of this article was arranged by Associate Editor Chenchen Liu. *(Corresponding author: Ramashish Gaurav.)*

Ramashish Gaurav, Sujata Sinha, Chunxiao Lin, Lingjia Liu, and Yang Yi are with the Department of Electrical and Computer Engineering, Virginia Tech, Alexandria, 22305, USA (e-mail: rgaurav@vt.edu; sujatasinha@vt.edu; chunxiaol@vt.edu; ljliu@vt.edu; yangyi8@vt.edu).

Terrence C. Stewart is with the National Research Council of Canada, University of Waterloo Collaboration Centre, Waterloo, N2L 3W8, Canada (e-mail: tcstewart@uwaterloo.ca).

This article has supplementary downloadable material available at https://doi.org/10.1109/TCASAI.2026.3658587, provided by the authors.

Digital Object Identifier 10.1109/TCASAI.2026.3658587

2996-6647 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

methods focus largely on deploying *spiking* networks on Loihi-2, not the *non-spiking* components.

The Loihi-2 chip has two types of computational elements embedded on it: (a) Lakemont cores and (b) Neuro-Cores. The Lakemont (LMT) cores are *low power* x86 micro-processors that support quantized integer operations, and the Neuro-Cores support the deployment of spiking networks. Therefore, one can use the embedded LMT cores to implement quantized versions of the non-spiking components in their SNN model, and implement the remaining spiking network on the Neuro-Cores. However, due to the *scarce* technical documentation to program the LMT cores, the researchers often do *not* leverage them, and rather deploy their non-spiking components on the off-chip CPUs / FPGAs, and spiking components on the on-chip Neuro-Cores. This practice incurs higher energy consumption and increased communication overheads. In other words, the researchers often do *not* implement their SNN model (comprising non-spiking and spiking components) *entirely* on the Loihi-2 chip, and rather choose to split the hardware backends [13].

In our recent work [14], we presented the detailed technical specifics to deploy and evaluate our Legendre-SNN *entirely* on Loihi-2. We achieved so by implementing the quantized non-spiking LDN on the LMT core, and the spiking network on the Neuro-Cores. Additionally, we also presented the technical documentation to program the LMT core (as part of the LDN's deployment). Note that the Legendre-SNN is designed for *univariate* Time-Series Classification (TSC) task, i.e., it classifies only one-dimensional input signals. The *univariate* Legendre-SNN can be adapted to handle *multivariate* time-series by adding one LDN per-dimension of the input signal - as presented in our recent Deep Legendre-SNN (DeepLSNN) work [15]. Note that apart from multiple LDNs per dimension of the input, the DeepLSNN's architecture also has multiple spiking hidden layers (hence "Deep"). In our DeepLSNN work [15], we have presented the *first-ever SNN benchmark* for spiking-TSC on 102 time-series datasets, where we found that DeepLSNN *outperformed* a well established deep learning benchmark model: LSTM-FCN [16], [17] on more than 31% of the 102 TSC datasets (on CPU/GPU); therein, we have also presented our insights into the Legendre-based SNNs' architectural design choices by theoretically investigating the LDN, where we demonstrated that *higher order* LDN is optimal for signals with *high frequency* components. With DeepLSNN establishing the prowess of Legendre-based SNNs on TSC tasks, the obvious next step is to demonstrate its implementation amenability on a neuromorphic hardware, and its real-world applications. Therefore, in this work, we extend our previous univariate "Legendre-SNN on Loihi-2" study [14] - by adapting and implementing the multivariate Legendre-SNN *entirely* on Loihi-2, and conducting experiments with multivariate TSC datasets. This work enables us to compare Legendre-based SNNs' results on *physical Loihi-2* with that of LSTM-FCN on *GPU*. We then explore multivariate Legendre-SNN's classification applications (on Loihi-2) in 5G wireless domain, where we consider the common task of Jamming Detection [18]. Our major contributions of this work, as well as a part of the contributions of our previous study [14] (included for completeness) are mentioned below:

A. Included Contributions

1) We present a quantized version of the LDN and implement it on the Loihi-2 embedded LMT core; thus, we add to the scarce documentation to program LMT core.
2) We implement the univariate Legendre-SNN on physical Loihi-2 chip and evaluate it on 15 univariate datasets.

B. Added Contributions

1) We adapt & implement our multivariate Legendre-SNN on Loihi-2; with experiments on 9 multivariate datasets. We also identify the limitation of a single Lakemont core to implement our reservoir (of multiple LDNs).
2) We demonstrate the application of our multivariate Legendre-SNN for 5G wireless jamming detection. We also present a demonstration video of jamming detection (with simulated signals) on the Loihi-2 board.
3) We conduct detailed energy-analysis of multivariate Legendre -SNN for jamming detection on Loihi-2 to bolster the cause for utilizing Neuromorphics for 5G-&-beyond.

II. Related Work & Background

Here, we present the existing spiking networks' related work on TSC, followed by the work on 5G jamming detection. A few of the recent works for spiking TSC are [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32]; where, [20], [27], [29], [31], [32] belong to the category of reservoir inspired spiking networks; [19], [21], [22], [24], [30] belong to the category of Deep Neural Network (DNN) inspired SNNs, and [23], [25], [26] belong to the category of Long Short-Term Memory (LSTM) inspired SNNs. With respect to the reservoir inspired spiking networks, [20] presented a *hybrid* network where a spiking reservoir acts like a feature extractor, whose output spikes are then fed to a conventional 2D Convolutional Neural Network (CNN) model for classification; [27], [29] on the other hand presented a *pure* SNN model, where [27]'s spiking reservoir resembles Liquid State Machine (LSM) [33], whereas our spiking reservoir [29] (deployed on Loihi-1 board) is inspired from the spiking implementation of the LDN [11]; finally, [31] and [32] used the novel Nonlinear Spiking Neural P systems [34] as a reservoir, along with conventional Machine Learning methods (e.g., Principal Component Analysis) in their model. Note that [28] too, proposed a novel kind of spiking network for TSC – based on evolving SNNs [35].

Coming to DNN inspired SNNs, [19] presented a spiking CNN model to process speech commands, and deployed their spiking network on first generation 48-chip SpiNNaker board; [21] presented a *hybrid* Convolutional SNN (for univariate and multivariate TSC), where ReLU convolutional layers are used to extract features, which are then encoded to spikes and later mapped to classes via a `Dense`-only (i.e., fully connected) SNN; [22] presented a more simple (but parameter heavy) `Dense`-only SNN for multivariate TSC; finally, [24] and [30] developed spiking CNNs to perform ECG classification. In regards to the LSTM inspired SNNs, [25] and [26] proposed spiking LSTMs, although, both of the works employed spiking

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

neurons that output *negative* spikes (which isn't biologically feasible) - to replace the `tanh` activation function in the LSTM; [23] too, proposed a spiking LSTM unit, but the dynamics of their spiking neurons aren't very clear. Note that [25] also implemented their spiking network on Loihi-1.

The training methodology of these aforementioned works range from conventional Machine Learning methods to ANN-to-SNN conversion strategies [36], [37] and ad-hoc training methods with surrogate derivatives [38], [39]. As can be seen, the existing works are a healthy mix of *hybrid* and *purely* spiking networks, out of which, only a few [19], [29], [25] deploy their spiking networks on early generations of neuromorphic hardware; however, none of them have been *deployed* and *evaluated* on the latest generation Loihi-2 board. Thus, this work also adds as a successor to the relatively smaller group of works for spiking TSC on a neuromorphic hardware. Note that we explain our previous related work on Legendre-SNN [10] later in the Sec II-C; also, later in Sec IV-B, we compare our results with those of [21], [10], [22], and [28]. Here, we underscore that the *hybrid* Convolutional SNN [21], as expected, rivals well against the deep learning models (observed in [21]), and consequently, against other spiking models too; this is because of the *trainable non-spiking* component in such heterogeneous SNNs, which also adds to the challenge of deploying these high-performing SNNs *entirely* on the neuromorphic hardware. Moreover, the (Convolutional) heterogeneity adds complexity to the architecture's design and training. Therefore, the priority should be to develop high-performing *homogeneous* spiking models due to their neuromorphic amenability. Our Legendre-based SNNs have a non-spiking LDN component, however, the LDN is *non-trainable* and is composed of *linear matrix operations*, which makes Legendre-based SNNs fairly simpler than other heterogeneous spiking models; and despite their simplicity, they too rival deep learning based models [15].

We now present the existing work on jamming detection in the context of 5G-&-Beyond. Jamming detection entails identifying corrupted signals transmitted by a malicious jammer over a wireless communication system; the corrupted signals can otherwise jam or disrupt the wireless communications. A few statistical approaches are [40], [41], [18], where [18] and [40] designed jamming detectors based on Generalized Likelihood Ratio Test. Other approaches, e.g., [42], [43], [44], [45], [46], [47], [48], [49], [50] utilize data driven machine or deep learning -based models to design jamming detectors. We highlight [44] & [46] as they are closely related to this work; [44] designed a Convolutional Autoencoder to distinguish *clean* signals from *jamming* signals (i.e., complex noise) in the In-phase & Quadrature (IQ) sample domain, and [46] proposed a Tiny-ML based approach - wherein they have trained and deployed a 1D-CNN on Raspberry Pi for jamming detection. We too detect *clean* and *jamming* signals, although - in a first (to our best knowledge) - unlike any of the existing works, we here deploy our jamming detection model on Loihi-2. We next lay down the background necessary for our work.

A. Loihi-2, KPSC Loihi-2 Board, and Lava

1) Loihi-2: Loihi-2 is Intel's latest 2nd-generation neuromorphic research chip, where one chip houses 128 Neuro-Cores (each supporting 8192 spiking units and connected by a network-on-chip) and 6 low-power x86 Lakemont (LMT) cores. The Neuro-Cores support asynchronous programmable CUrrent-BAsed (CUBA) spiking neurons and operations with binary & 32-bit graded spikes. The LMT cores manage spike-based communication and assist with network configuration and monitoring, apart from executing standard C code for data I/O. Note that the LMT cores support 32-bit *quantized* operations, which implies that one can leverage these embedded micro-processors (instead of off-chip CPUs) to implement quantized versions of the non-spiking operations (in their SNN models).

2) KPSC Loihi-2 Board: Kapoho Point Single Chip (KPSC) Loihi-2 board system is built with (i) a mezzanine card housing a single Loihi-2 chip, (ii) a System-on-Module (SoM) that interfaces with the Loihi-2 chip on the mezzanine card, and (iii) a baseboard that hosts the SoM and the mezzanine card on both sides each. The baseboard houses an Ethernet port, which one can connect to a router and access the Loihi-2 chip through their Linux PC (also connected to the router). We present more details in the demonstration Sec IV-F later.

3) Lava: Lava is Intel's open-source software suite (available at: https://lava-nc.org/) to program the Loihi-2 chips; the suite consists of multiple libraries, e.g.,`lava`, `lava-dl`, etc., that help in *building* and *deploying* various kinds of spiking networks on Loihi-2. To build Loihi-2 deployable versions of Legendre-SNN [10], we use *two* core programming paradigms of Lava: `Process` & `ProcessModel`. A `Process` is a Lava class that defines the *interface* of a network's component, while a `ProcessModel` is another Lava class that defines the *hardware-dependent implementation* of a `Process`; i.e., a Lava `Process` is deployable on CPU, GPU, & Loihi. Note that *one* `Process` can have *multiple* `ProcessModels` ; and a `Process` is mapped to *one* of its `ProcessModels` during the runtime - based on the desired backend of that `Process`.

Lava supports three different types of `ProcessModels`: `PyLoihiProcessModel` (for Python implementations on the CPU), `CLoihiProcessModel` (for C implementations on LMT cores), and `NcProcessModel` (for NxCore implementations on Neuro-Cores); we use `CLoihiProcessModel` to implement our quantized non-spiking LDN via a C program on the LMT core. The `lava-dl` library is used to build and directly train the SNN models on CPU/GPU, and it also assists in *conversion* of a trained SNN model to its Loihi-2 deployable format via its `netx` API. Note that Lava provides a *simulation* backend too - for the *physical* Loihi-2 chip, and one can easily switch between these two backends using `lava`'s APIs.

B. Legendre Delay Network (LDN)

Legendre Delay Network (LDN) is a neural approximation of a Linear Time Invariant system implementing a delay of - say $\theta \geq 0$ time-steps; the LDN can be realized via regular linear matrix operations, as well as, via spiking neurons by following the Neural Engineering Framework theory [11], [51]. For a given *univariate* input $u[t]$ (in discrete time-steps), the LDN outputs $y[t] \approx u[t-\theta]$ (i.e., delayed by θ), by following a linear

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

system of discrete-time state-equations (Eqs (1a), (1b)):

$$x[t+1] = \mathbf{A}x[t] + \mathbf{B}u[t] \tag{1a}$$

$$y[t] = \mathbf{C}x[t] + \mathbf{D}u[t] \tag{1b}$$

where $x[t] \in \mathbb{R}^{d\times 1}$ is the LDN's state-vector, $u[t] \in \mathbb{R}$ and $y[t] \in \mathbb{R}$ are the system's input and output respectively; $\mathbf{A} \in \mathbb{R}^{d\times d}$, $\mathbf{B} \in \mathbb{R}^{d\times 1}$, $\mathbf{C} \in \mathbb{R}^{1\times d}$, and $\mathbf{D} \in \mathbb{R}$ are the state-matrices (values defined in Appx B of Suppl doc.). In Legendre-SNN [10], only the Eq (1a) is used to compute the d-dimensional $x[t]$ from the input signal $u[t]$; note that the *state-vector* $x[t]$ is also referred to as *temporal feature-vector*. Also from the Appx B, it is easy to infer that an LDN is jointly characterized by (d, θ).

C. Legendre Spiking Neural Network (Legendre-SNN) [10]

We next explain the architecture of our *univariate* Legendre-SNN (see Fig. 1) proposed in [10] that we adapt in this work to enable its Loihi-2 deployment. As can be seen in the Fig. 1, the Legendre-SNN is composed of 1 -dimensional input (**INP**) node (feeding a signal $u[t]$), one LDN (Eq (1a)) acting as a *static* reservoir, one encoding (**ENC**) layer, one spiking hidden (**HDN**) layer, and an output (**OTP**) layer; both the **HDN** & **OTP** layers are all-to-all connections and are *trainable*, and all the spiking neurons in **ENC** & **HDN** layers are Integrate & Fire (IF) neurons. The *static* LDN extracts d-dimensional temporal features $x[t]$ (from the input signal $u[t]$) that are rate encoded to binary spikes via the **ENC** layer neurons. Note that the **ENC** layer is composed of *two-neuron encoder* systems, where a pair of IF neurons encode *each* dimension of $x[t]$. Such an encoding system is needed because unlike images (where each pixel ≥ 0), each scalar / dimension in $x[t]$ can either be positive or negative at any given time-step t. Therefore, to encode *one* dimension of $x[t]$, we need *two* neurons, where one is sensitive to positive values and another is sensitive to negative values; note that in such encoding pairs, only *one* of the two neurons spike at a single time-step t (if the spiking threshold conditions are met). In [10], we trained the Legendre-SNN using Surrogate Gradient Descent [39], and the class inference was done based on the maximally accumulated voltage trace of the **OTP** layer units (i.e., they do not spike). Coming to the number of neurons in the Legendre-SNN, the **ENC** and **HDN** layers have $2 \times d$ (henceforth $2d$) and $3 \times d$ (henceforth $3d$) IF neurons respectively; thus, the number of trainable connections are $2d \times 3d + 3d \times c = 3d \times (2d + c)$, where c is the number of classes.

III. Methods

In this section, we explain our methods to *adapt* and *map* the Legendre-SNN [10] *entirely* on the Loihi-2 neuromorphic chip, as well as its training procedure on CPU/GPU. In [14], we have presented our approach to adapt and map the *univariate* Legendre-SNN entirely on Loihi-2; therefore, in this work, we include our approach and extend it to adapt & implement the *multivariate* Legendre-SNN on Loihi-2. Note that the multivariate Legendre-SNN is a generalized & scaled up version of the univariate Legendre-SNN, i.e., one can achieve univariate TSC with multivariate Legendre-SNN too. Therefore, for the remainder of this work, we ignore the operational distinction between multivariate & univariate Legendre-SNN.

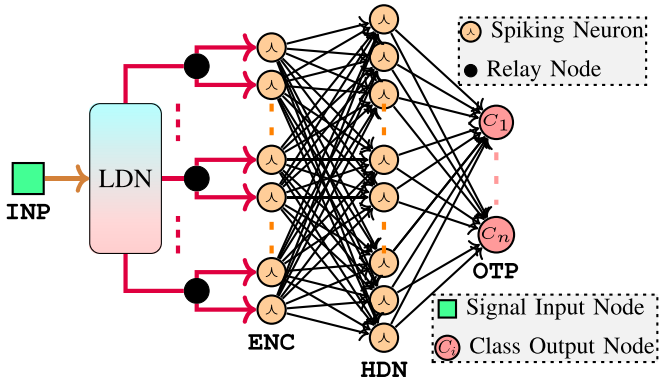


Fig. 1. Our *Legendre-SNN* model from [10]. **INP** is composed of one input node for univariate signal. The LDN computes a d-dimensional feature-vector $x[t]$ (from input), whose each scalar value is duplicated and encoded to binary spikes via **ENC**. Connections from **ENC** to **HDN** to **OTP** are all-to-all `Dense`. The **OTP** computes classes C_i; all the "spiking neurons" are integrate & fire (IF) neurons, and only the black connections post **ENC** layer are trainable.

As mentioned before, mapping such an SNN model (Legendre -SNN in our case) composed of *non-spiking* & *spiking* components entirely on Loihi-2 is *not* trivial; often, the researchers *split* their SNN model, and deploy the non-spiking components on power-intensive (off-chip) CPUs and spiking components on Loihi-2's Neuro-Cores [13]. This unfortunately brings *latency* in communication between the non-spiking and spiking components, as well as, *underutilizes* the best of energy-efficiency offered by Loihi-2. To avoid this untoward setting, we use the Loihi-2's on-chip embedded LMT core to implement the non-spiking components. However, the technical specifics to program the LMT cores is scarce, therefore, in this section, we present the technicalities of implementing the non-spiking LDN on the embedded LMT core. Our presented methods and technical details can be easily leveraged by other neuromorphic researchers to map their specific non-spiking components on LMT; thus, implementing their similar (hybrid) SNN models *entirely* on Loihi-2. Note that, keeping in mind of the eventual deployment and evaluation of Legendre-SNN on Loihi-2, we first *adapt* it to meet the Loihi-2's design constraints; therefore, we next present our Loihi-2 adaptation methods.

A. Loihi-2 adaptation of Legendre-SNN

As seen in Fig. 1, the Legendre-SNN has multiple distinct components, which must be separately accounted for while mapping to Loihi-2. In the original Legendre-SNN [10], the LDN receives continuous-valued input $u[t]$ from **INP**, and the **ENC** receives continuous-valued $x[t]$ from the LDN. The **HDN** and **OTP** layers receive and operate on binary spikes, where the **OTP** layer infers classes based on the maximally accumulated voltage. To map such a *heterogeneous* SNN model on Loihi-2, we next present our methods to map/adapt its components.

1) Adaptation of non-spiking LDN: Since the LMT cores on Loihi-2 execute quantized 32-bit operations, we start with our approach to quantize the LDN. A common way to quantize a

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

floating-point number is by scaling it with a power of 2 (say $\mathcal{P}$) and then clipping away the fractional part, thus, the remaining integer represents a quantized approximation of the original floating-point number. For the given floating-point signal $u[t]$, let $\bar{u}[t]$ represent its quantized approximation, i.e., we denote the operation of scaling $u[t]$ and clipping away the fractional part as $\bar{u}[t] = \langle \mathcal{P} \times u[t] \rangle$ ($\langle . \rangle$ denotes rounding to the nearest integer). Note that in the LDN's Eq (1a) (used in the Legendre-SNN), all the matrices and vectors are floating-points, therefore, we quantize them too, i.e.:

$$\bar{\mathbf{A}} = \langle \mathcal{P} \times \mathbf{A} \rangle, \quad \bar{\mathbf{B}} = \langle \mathcal{P} \times \mathbf{B} \rangle, \quad \text{and} \quad \bar{x}[t] = \langle \mathcal{P} \times x[t] \rangle$$

Thus, we can write quantized representation of Eq (1a) as:

$$\bar{\bar{x}}[t+1] = \bar{\mathbf{A}}\bar{x}[t] + \bar{\mathbf{B}}\bar{u}[t] \tag{2}$$

Note that $\bar{\bar{x}}[t+1]$ has $\mathcal{P}$'s contribution twice, therefore, for the next time-step's operation, we scale it back as: $\bar{x}[t+1] = \left\lceil \frac{\bar{\bar{x}}[t+1]}{\mathcal{P}} \right\rceil$ ($\lceil . \rceil$ is ceil operation). Thus, we can now implement the quantized version of the LDN with INT32-bit operations on the LMT cores of Loihi-2. Note that this quantized LDN can also be implemented on CPU/GPU - which we do during training of our Loihi-2 adapted versions of the Legendre-SNN.

2) Adaptation of ***ENC****,* ***HDN*** *&* ***OTP*** *Layers:* The **ENC** layer is supposed to rate encode the quantized temporal features $\bar{x}[t]$, therefore, we quantize the **ENC** IF neurons' equations by using our aforementioned quantization approach; the quantized **ENC** IF neuron equations are in the Appx C.1 of Suppl doc. In [10], the original Legendre-SNN had IF neurons in the **HDN** layer and *no* spiking neurons (rather decaying voltage accumulators) in the **OTP** layer. To deploy the **HDN** & **OTP** layers on Loihi-2's Neuro-Cores, we build both of them with Loihi-2 supported CUBA neurons; their equations are in Appx C.2 of Suppl doc.

B. Loihi-2-adapted multivariate Legendre-SNN

We now integrate the aforementioned Loihi-2 adapted layers to build the architecture of our multivariate Legendre-SNN (mv-LSNN henceforth) that is *entirely* deployable on Loihi-2. Note that the mv-LSNN is also deployable on CPU/GPU - whereupon we train it, and then port it to Loihi-2 for inference.

Fig. 2 shows the architecture of our Loihi-2 -adapted mv-LSNN model. The **INP** layer has K input nodes (for a K-dim signal), and the static reservoir (**RES**) comprises K quantized LDNs - each connected one-to-one to input nodes in **INP**; note that **INP** feeds quantized input signal to **RES**. Each quantized LDN in **RES** extracts a d-dim feature-vector $\bar{x}[t]$ from the *individual* dimensions of the K-dim input signal; post which, all the 'K' $\bar{x}[t]$ vectors are concatenated, and then encoded to binary spikes by the quantized **ENC** - that constitutes of $K \times 2d$ (henceforth, $2Kd$) number of IF neurons. Note that similar to original Legendre-SNN, we use *two-neuron encoder* system. The trainable **HDN** & **OTP** layers have $K \times 3d$ (henceforth, $3Kd$) and c CUBA neurons respectively (c is #classes). Note that distinct from the original Legendre-SNN (Fig. 1), we have added a dedicated classification (**CLS**) layer in our mv-LSNN (Fig. 2) to infer classes from the output spike-trains of the **OTP** layer; the nodes in **CLS** layer are one-to-one connected to the

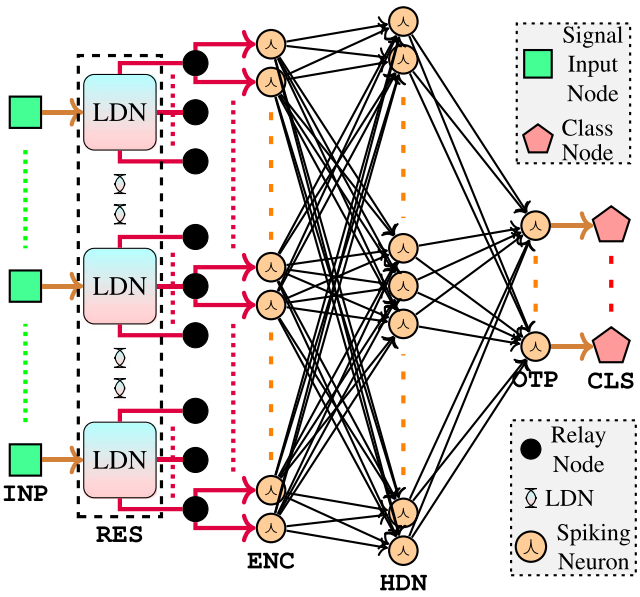


Fig. 2. Our *mv-LSNN* model for multivariate signals. **INP** is composed of K input nodes, each corresponding to a dimension of the K-dimensional signal. **RES** is composed of K quantized LDNs, each connected to one dimension of the multivariate input. The quantized state-vectors $\bar{x}[t]$ from each LDN are concatenated, each scalar is then duplicated and sent to the quantized **ENC** to be encoded to spikes. The spiking neurons in **ENC** are IF, and in **HDN** & **OTP** are CUBA. Connections from **ENC** to **HDN** to **OTP** are `Dense` all-to-all; **OTP** to **CLS** are one-to-one. **CLS** infers classes on the **OTP** neurons' firing rates.

OTP layer neurons, and the inferred classes correspond to the index of the maximally firing **OTP** neuron. Also note that only the black connections from **ENC** to **HDN** and from **HDN** to **OTP** are trainable; thus, the total number of trainable weights in mv-LSNN are $2Kd \times 3Kd + 3Kd \times c = 3Kd \times (2Kd + c)$. It is easy to observe that for the univariate input signal case (i.e., $K = 1$), the number of trainable weights in mv-LSNN is same as the number of weights in the original Legendre-SNN.

C. Training mv-LSNN

Now that we have described the architecture of our Loihi-2 adapted mv-LSNN, we next explain our method to train it on CPU/GPU. We begin by identifying the hardware backends on which the individual components of mv-LSNN are deployed while training. As can be seen in Fig. 3, the processing of input signal $\bar{u}[t]$ to temporal features $\bar{x}[t]$ by the reservoir, followed by encoding of $\bar{x}[t]$ to spikes - happens on CPU, i.e., the *static* block of **INP**, **RES**, & **ENC** layers are deployed on the CPU. The spikes are then fed to the *trainable* block of **HDN** & **OTP** layers, which can be deployed / trained on CPU/GPU. Finally, the inference of classes from the output spike train happens in the *static* block of **CLS** layer deployed on CPU. We train the block of **HDN** & **OTP** layers by building a network out of them - using `lava-dl`'s `slayer` API; we call this CPU/GPU trainable network as *SlayerSNN* (i.e., **HDN**→**OTP**). The *SlayerSNN* is trained by computing `SpikeRate` loss (defined in `slayer`) over the **OTP** spike rate; we use Adam [52] to train weights. Note that while training, we tune the CUBA neurons' *current* and *voltage* decay hyper-parameters too. Also, during

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

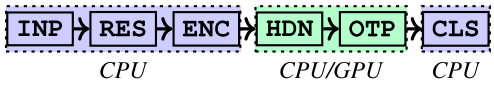


Fig. 3. H/w backends for *training* Loihi-2 -adapted mv-LSNN on CPU/GPU.

each training epoch, we evaluate the trained mv-LSNN on test set, and save the best-performing corresponding *SlayerSNN*; this saved network is then integrated with the **RES** and **ENC** layers for a later inference on Loihi-2 – as explained next.

D. Evaluating trained mv-LSNN on Physical Loihi-2

As mentioned in the Sec II-A3, one needs to build their SNN model with Lava `Processes` to deploy them on the *physical* Loihi-2 hardware, as well as on Loihi-2's software *simulation* running on CPU. However, before we explain our Lava `Processes` of the (trained) mv-LSNN model, we briefly present the Loihi-2 backends of its individual components; this will greatly aid us in our explanation of the mv-LSNN's Lava `Processes`. As can be seen in the Fig. 4(a), while evaluating the *trained* mv-LSNN on *physical* Loihi-2, the **INP** layer is deployed on CPU, which inputs the quantized signal (from CPU) to the block of **RES** & **ENC** layers, which are deployed on the LMT core. The spikes from **ENC** are fed to the trained *SlayerSNN*, i.e., the **HDN** & **OTP** layers, which are deployed on Neuro-Cores. The **CLS** layer is deployed on CPU where it receives spikes from the **OTP** neurons and infers the predicted class based on the maximally firing rate (of **OTP** neurons).

Next, as can be seen in the Fig. 4(b), the Loihi-2 deployable version of mv-LSNN is built with 4 main Lava `Processes`: *InpSigToRes*, *ResEncToSpk*, *NetxSNN*, and *OutSpkToCls* - each responsible for implementing one or more layers of the mv-LSNN (see color coding in Fig. 4(a)). Fig. 4(b) also shows the respective hardware backends of the `Processes` - for mv-LSNN's deployment on Loihi-2's *simulation*, as well as on the *physical* Loihi-2 chip. Note that since Loihi-2's *simulation* runs on CPU, all the Lava `Processes` of mv-LSNN (when evaluated on Loihi-2's *simulation*) run on CPU-only. However, when we evaluate mv-LSNN on *physical* Loihi-2, the case is different; the Lava `Processes`: *InpSigToRes* (i.e., **INP**) and *OutSpkToCls* (i.e., **CLS**) are by-design deployed on CPU because they are inputs and outputs respectively, whereas, the `Processes` *ResEncToSpk* (i.e., **RES** & **ENC**) and *NetxSNN* (i.e., **HDN** & **OTP**) are deployed on *physical* Loihi-2's LMT core and Neuro-Cores respectively. Note that the *NetxSNN* is obtained from the best performing *SlayerSNN* (saved earlier during training) using Lava's `netx` API. Consistent with Fig. 4(a), the *InpSigToRes* `Process` inputs quantized signal to the *ResEncToSpk* `Process` that extracts temporal features and encodes them to binary spikes; these spikes are then fed to the *NetxSNN* Process, whose output spikes are sent to the *OutSpkToCls* `Process` that infers classes. Note: as stated in Sec II-A3, Lava `Processes` define only the *interface* of a network's components; one must write their corresponding hardware-dependent `ProcessModels` to implement their intended tasks. Therefore, we explain our `ProcessModels` of the mv-LSNN's `Processes` in the Appx D of Suppl doc.

E. Programming LDN & **ENC** on LMT Core

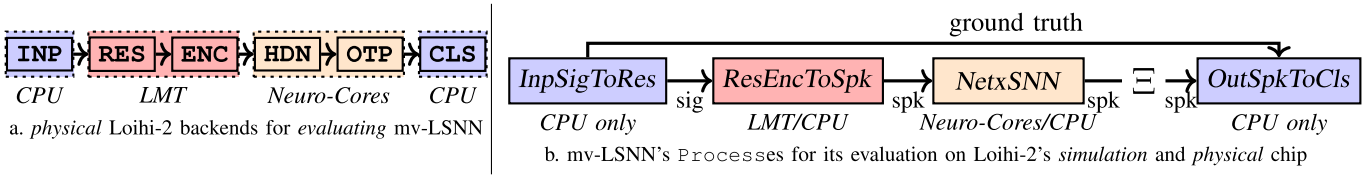
Here we present the details of one of our major contributions in this work: the technical specifics to program the Loihi-2 - embedded LMT cores. We begin by noting that the technical documentation to program Loihi-2 via Lava library is available at https://lava-nc.org/. There, the documentation to program Loihi-2's Neuro-Cores is well presented, however, there's a dearth of documentation to program the on-chip LMT cores – due to which, the researchers are either *unaware* of such an on-chip computational resource or are *unable* to leverage it; thereby, implementing their non-spiking and encoding operations on the off-chip energy-intensive CPUs (with subsequent spiking operations on Loihi-2's Neuro-Cores of-course). Therefore, in light of LMT's low power consumption on Loihi-2, we propose to leverage it for implementing quantized versions of the non-spiking and encoding operations – within the hardware constraints of LMT. Here, by means of implementing our quantized LDN(s) - constituting the **RES** layer, and the **ENC** layer on LMT, we present the *technical specifics* to program the on-chip LMT core – these *specifics* can be easily leveraged by other neuromorphic researchers to implement their non-spiking components right on the Loihi-2's embedded LMT cores; thereby, enabling the implementation of their SNN with non-spiking and spiking components *entirely* on Loihi-2 and profiting from the best of its offered energy-efficiency.

To program the LDN(s) (i.e., **RES** layer) and **ENC** layer on Loihi-2's LMT core, we define the *ResEncToSpk* Process, which receives a quantized input signal from the *InpSigToRes* `Process` and outputs binary spikes to *NetxSNN* `Process` (see Fig. 4(b)). Recall from the Sec II-A3, that to implement a `Process` on LMT core, one needs to build its corresponding `CLoihiProcessModel` written in C. Therefore, to execute *ResEncToSpk* Process on Loihi-2's LMT, we create its corresponding `CLoihiProcessModel` named: *CResEncToSpkOnLmtModel*. Owing to space constraints, we present the C code of our *CResEncToSpkOnLmtModel* in Algorithm 1 - in Appx E of Suppl doc, where we describe the technical details to implement the non-spiking quantized LDN(s) (in **RES**) and the **ENC** layer on an LMT core. There, we also describe the *foundational concepts* and Lava APIs to receive inputs from a CPU bound `Process` and send outputs (from an LMT bound Process) to a Neuro-Core bound `Process`. Thus, other researchers can leverage our technical findings and adapt our C program to implement their own non-spiking components and encoding methods on the Loihi-2 embedded LMT core.

F. 5G Wireless Jamming Detection Task

We now unveil another key contribution: detecting jamming attacks in 5G wireless systems. Among the various threats to wireless reliability, deliberate signal interference remains one of the most disruptive attacks. A sophisticated adversary may target only *specific* components of the signal in order to remain stealthy, while still degrading the system performance. Note

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.



a. *physical* Loihi-2 backends for *evaluating* mv-LSNN

b. mv-LSNN's `Processes` for its evaluation on Loihi-2's *simulation* and *physical* chip

Fig. 4. *mv-LSNN*'s Loihi-2 backends and `Processes`. Note the color coding between (a) & (b). 'sig' and 'spk' in (b) are 'signal' and 'spikes' respectively; Ξ in (b) is an adapter `Process` to adapt spikes from Neuro-Core to CPU. For evaluation on Loihi-2's *simulation*, the `Processes`' backends are all CPU.

that a transmitted signal is composed of *pilot symbols* and *data symbols*. These *pilot symbols*, known *a priori* to both transmitter and receiver, are strategically embedded within the transmitted signal to enable crucial communication functions. The *data symbols* are the modulated representations of the actual information sent over the communication channel. Modern wireless systems depend heavily on the pilot symbols for accurate channel estimation [53]. By *corrupting* only these pilot symbols, a jammer can impair the receiver's ability to estimate the channel—without affecting the energy distribution or the apparent structure of the remaining signal. Consequently, the jammer makes *substantial* impact with a *subtle* interference. This form of attack presents a unique challenge for detection systems that rely on traditional signal-level monitoring.

Pilot symbols are also used for symbol prediction tasks; e.g., in [54] the authors use the received pilot symbols to train the weights of their model to predict the transmitted data symbols. Therefore, even a subtle distortion in pilot symbols can result in the corruption of weight training, causing significant degradation in symbol prediction performance. Thus, the attacker's goal is to *maximize* the receiver's symbol prediction *error* without directly interfering with the data-bearing symbols. Our dataset captures this setup and supports a binary classification task. In short, we focus on pilot jamming in this work; where, we frame the detection task as a binary classification problem - in which the receiver classifies (based solely on the observed signal) that whether the transmission was *clean* or subjected to a pilot *jamming* attack. A *clean signal* refers to a legitimate transmission from the Base Station (BS) to the receiver User Terminal (UT), while a *jammed signal* is one in which the pilot signals have been deliberately corrupted by an adversary.

1) System Setup for WirelessJamming Dataset: Our dataset generation scenario involves a legitimate transmitter-receiver pair and a jammer operating as an over-the-air adversary (see Fig. 5). Both legitimate and adversarial transmitters share the same physical-layer configuration. On the transmitter side, each binary input passes through a standard 5G NR physical-layer chain, including channel coding, modulation, and precoding. The resulting signal X is then transmitted over the channel. At receiver, a Windowed Echo State Network (WESN) [54] performs symbol detection on the received signal Y; it is followed by demapping and decoding to recover the transmitted bits to compute Bit Error Rate (BER). During normal operation (BS to UT), the received signal $Y_{\text{legitimate}}$ at the UT is given by:

$$Y_{\text{legitimate}} = HX + Z \tag{3}$$

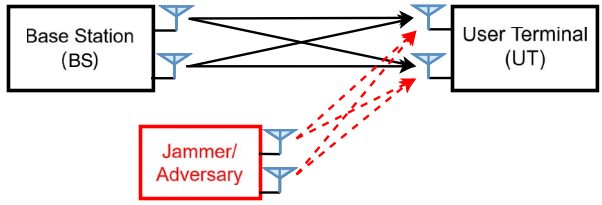


Fig. 5. Our setup for creating the *WirelessJamming* dataset.

where $H \in \mathbb{C}^{2\times 2}$ is the complex Clustered Delay Line (CDL) MIMO channel matrix, X is the transmitted signal, and $Z \sim \mathcal{CN}(0, \sigma_Z^2 I)$ is a circularly symmetric complex Gaussian noise. In the presence of an adversary transmitting jamming signals, the received signal $Y_{\text{adversary}}$ at UT becomes:

$$Y_{\text{adversary}} = HX + Z + J \tag{4}$$

where $J \sim \mathcal{CN}(0, \sigma_J^2 I)$ is the jamming signal added by the adversary to the pilot symbols only. Here, we assume that the adversary knows the location of the pilots.

In our experiment setup, we consider a fixed energy-per-bit to noise power spectral density ratio (E_b/N_0) between the BS and UT; the jammer simply intends to increase the E_b/N_0 of the pilots. Our goal on the other hand is to learn a function:

$$f : \mathbb{C}^{(N_{\text{pilots}}+N_{\text{data}})\times(n_{\text{fft}}+l_{\text{cp}})} \rightarrow \{0, 1\}, \tag{5}$$

where n_{fft} is number of subcarriers, l_{cp} is cyclic prefix length, and N_{pilots} & N_{data} denote the number of pilot & data symbols, respectively. The function f maps the received signal Y to a binary label indicating if the signal is clean (i.e., 0) or jammed (i.e., 1). To simulate varying levels of adversarial interference, we increase the E_b/N_0 between the Jammer and UT from 0 dB to 5 dB with a step size of 1 dB (E_b/N_0 between BS and UT is kept constant). These values are chosen based on our empirical observations that the BER rises with increasing jamming strength and saturates to 0.50 around 5 dB, indicating complete communication breakdown. This setup enables us to systematically evaluate the sensitivity of our detection models to increasingly disruptive jamming, while maintaining a constant baseline signal quality from the legitimate BS to UT. We present the values of N_{pilots}, N_{data}, n_{fft}, & l_{cp}, as well as more technical details of creating our dataset (i.e., *WirelessJamming*) in the Appx F of Suppl doc. That said, our *WirelessJamming* dataset consists of 1000 samples each of *clean* and *jamming* signals (each 1016 time-steps long); for a challenging setting, we randomly divide the whole set of 2000 samples into 60/40 % train-test sets.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

IV. Experiments & Results

We next explain our chosen datasets, experiments, results, and video demonstration of jamming detection on the Loihi-2 board. In this work, we have conducted experiments on 9 *multivariate* TSC datasets; we supplement these with our previous experiments on 15 *univariate* TSC datasets from [14].

A. Datasets' Details

The 9 multivariate datasets' signal dimension are $\in [2, 6]$, number of classes are $\in [2, 8]$, and signal duration are $\in [30, 2500]$ time-steps. The 15 univariate dataset's number of classes are $\in [2, 5]$ and signal duration are $\in [140, 900]$ time-steps. A detailed tabular description of all the 24 datasets is provided in Appx G of Suppl doc. Note that our chosen datasets belong to a wide range of application domains: ECG, EEG, Sensor, Spectrograph, Motion, Device, Human Activity Recognition, & 5G Wireless. All the datasets except our *WirelessJamming* are available at https://timeseriesclassification.com/dataset.php; we have made the 5G *WirelessJamming* dataset public in this work.

B. Training & Evaluation Details of mv-LSNN on CPU/GPU

Our mv-LSNN's training methodology (on CPU/GPU) is already explained in Sec III-C; here, we complement it with the details of our experimental procedure. The tunable parameters of LDN are d & θ; recollect that the number of neurons in **ENC** & **HDN** layers are $2Kd$ and $3Kd$ respectively (K is the dimension of input) - thus, LDN's d and dataset's K determine the mv-LSNN's architecture. Note that, we also tune the mv-LSNN's CUBA neurons' current and voltage decays, i.e., τ_{cur} and τ_{vol} respectively; we provide the values of K, d, θ, τ_{cur}, τ_{vol} hyper-parameters in the Appx H of Supp doc. The batch size and number of training epochs for each dataset can be found in our code. Note that we set the quantization scaling factor $\mathcal{P} = 4096$ to well represent our operations in the quantized **RES** and **ENC** within 32-bit; also note that we halve the initial learning rate $\eta = 0.001$ after every 40th training epoch.

While training mv-LSNN on CPU/GPU - for each dataset, we conduct grid search on all the tunable hyper-parameters d, θ, τ_{cur}, & τ_{vol}; henceforth, let κ_m denote one combination of $(d_i, \theta_j, \tau_{\text{cur}_k}, \tau_{\text{vol}_l})$ among the grid-search combinations. For each κ_m, we run the specific mv-LSNN for 5 seeds s_n. As mentioned earlier in the Sec III-C, at the completion of each training experiment for κ_m & s_n, we save a trained mv-LSNN (*SlayerSNN* to be precise) that corresponds to the highest test accuracy (over all epochs) on CPU/GPU – say $slyr_acc_{s_n}^{\kappa_m}$. Next, we port that saved mv-LSNN to Loihi-2's *simulation* (running on CPU), and obtain the corresponding test accuracy – $l2sim_acc_{s_n}^{\kappa_m}$. From these two backends' test accuracies, we compute the *best* and *mean* accuracies on CPU/GPU as: $slyr_best = \max_{\kappa_m, s_n} (slyr_acc_{s_n}^{\kappa_m})$, and $slyr_mean = \max_{\kappa_m} (\text{mean}_{s_n} (slyr_acc_{s_n}^{\kappa_m}))$ respectively; and on Loihi-2's *simulation* as: $l2sim_best = \max_{\kappa_m, s_n} (l2sim_acc_{s_n}^{\kappa_m})$, and $l2sim_mean = \max_{\kappa_m} (\text{mean}_{s_n} (l2sim_acc_{s_n}^{\kappa_m}))$ respectively – reported in Table I.

TABLE I
MV-LSNN'S ACCURACY (IN %) ON UNI- & MULTI-VARIATE DATASETS

Datasets	mv-LSNN's Accuracies					Others	
	slyr _best	*slyr _mean*	*l2sim _best*	*l2sim _mean*	*l2hw _best*	*LSTM -FCN*	*Spk -Acc*
AtrialFibrillation	**73.33**	**54.66**	**73.33**	**50.67**	**66.67**	33.30	46.60$^{\mp}$
BasicMotions	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	87.50	100.0$^{\mp}$
Coffee	**100.0**	95.71	**100.0**	90.71	92.86	100.0	100.0$^{\mp}$
Computers	71.60	68.56	69.60	67.68	70.40	86.00	-
ECG5000	95.51	94.95	95.31	94.83	95.33	94.73	98.49$^{\perp}$
ERing	90.00	84.96	87.78	86.27	88.15	94.10	26.60$^{\mp}$
Earthquakes	79.14	76.26	76.98	75.40	77.70	83.54	80.43$^{\perp}$
Epilepsy	**92.75**	**91.30**	**92.75**	**90.87**	**92.03**	73.20	98.60$^{\mp}$
FordA	91.29	90.86	90.98	90.71	90.61	92.72	93.56$^{\perp}$
FordB	81.61	79.73	80.12	78.57	77.90	91.80	82.72$^{\perp}$
Lightning2	78.69	72.13	78.69	71.80	68.85	80.33	80.33$^{\mp}$
Meat	**100.0**	**95.33**	**96.67**	**92.00**	**98.33**	91.67	-
OliveOil	66.67	46.68	63.33	43.33	46.68	86.67	-
RacketSports	82.90	79.61	78.95	77.37	80.26	84.20	87.00$^{\curlyvee}$
RefrigerationDevices	54.40	48.75	53.33	49.39	50.40	58.31	-
SelfRegulationSCP1	**90.80**	87.92	90.10	89.08	59.04	90.80	-
StandWalkJump	**66.67**	**55.56**	**66.67**	**53.33**	-	40.00	-
ToeSegmentation1	93.86	91.75	92.98	91.49	92.11	98.25	-
ToeSegmentation2	**93.85**	88.92	**93.85**	88.31	90.77	93.08	83.00$^{\cup}$
UWaveGestureLibrary	**89.69**	**87.38**	**89.38**	**87.50**	**88.43**	85.90	89.10$^{\mp}$
Wafer	97.96	97.70	98.26	97.91	98.38	99.92	100.0$^{\mp}$
Worms	**80.52**	**75.32**	**79.22**	**75.06**	**75.32**	66.85	-
WormsTwoClass	**87.01**	**84.68**	**85.71**	**84.94**	**84.42**	79.56	-

This table presents results over 23 datasets, for remaining *WirelessJamming*, mv-LSNN's results are in Tab. II. For *ECG5000*, all the results except that of LSTM-FCN, are obtained after binarizing the dataset; therefore, we do not compare ours with LSTM-FCN. $slyr_*$, $l2sim_*$, and $l2hw_*$ denote mv-LSNN's accuracies on CPU/GPU, Loihi-2's *simulation* (on CPU) and the *physical* Loihi-2 board. LSTM-FCN and Spk-Acc denote accuracies of LSTM-FCN [16], [17] and other spiking networks ($\mp \Rightarrow$ [21], $\perp \Rightarrow$ [10], $\curlyvee \Rightarrow$ [22], $\cup \Rightarrow$ [28]); **bold** and underline denote mv-LSNN's accuracies higher than that of LSTM-FCN and other spiking networks respectively.

C. Evaluation Details of mv-LSNN on Physical Loihi-2

To evaluate mv-LSNN on the *physical* Loihi-2 chip for each of the 24 datasets, we consider all those mv-LSNN models that correspond to the dataset's $l2sim_best$ accuracy (as computed in Sec IV-B); as well as, models whose $l2sim_acc_{s_n}^{\kappa_m}$ is within 2% or 3% absolute margin of the dataset's $l2sim_best$ accuracy. We then execute our chosen models (for each dataset) on the entire test set on the *physical* Loihi-2 board - either on Intel's INRC cloud or locally with us. Finally, for each dataset, upon completing this inference phase on *physical* Loihi-2, we compute the maximum of all the mv-LSNN models' achieved accuracies and name it as $l2hw_best$- that we report in the Table I. Note that for the dataset *StandWalkJump*, we could not execute its mv-LSNN models on Loihi-2 chip because of its very long signal duration i.e., 2500 time-steps - due to which the *physical* Loihi-2 execution threw runtime errors.

D. Results Comparison and Analysis

We now analyze our results in the Table I, where we present our mv-LSNN's best and mean results obtained on CPU/GPU (i.e., $slyr_best$ & $slyr_mean$), on Loihi-2's *simulation* (i.e., $l2sim_best$ & $l2sim_mean$), and on *physical* Loihi-2 (i.e., $l2hw_best$); we compare our results with LSTM-FCN [16],

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

[17] and other spiking networks ("Spk-Acc" column). LSTM-FCN is a popular *non-spiking* benchmark model (for univariate [16] & multivariate [17] TSC); it is composed of LSTM layer and multiple 1D Conv layers (with pooling and batch-norm), thus, it is quite *complex* compared to our *simple* mv-LSNN.

As can be seen in Table I, considering mv-LSNN's best and mean performance on CPU/GPU and on Loihi-2's *simulation*, it outperforms LSTM-FCN on *at least* 8 of 23 datasets; and on *physical* Loihi-2, mv-LSNN outperforms LSTM-FCN on 7 of 23 datasets. With respect to the spiking networks, note that our compared works fall in two categories: (1) ANN-SNN blended models that have *trainable ANN & spiking* components, e.g., [21] (denoted by $\mp$ in Table I) has *trainable Conv layers with ReLU* neurons, and (2) Pure-SNNs that have *only trainable spiking* layers, e.g., [10], [22], [28] - denoted by $\perp$, $\curlyvee$, $\cup$ respectively in the Table I. Compared to the ReLU Conv-SNN of [21] ($\mp$), our purely spiking mv-LSNN outperforms it on *at least* 3 of 8 datasets. With respect to pure SNNs, our mv-LSNN outperforms [28] ($\cup$) by a large margin, however it loses to its predecessor [10] ($\perp$) because we trained and tuned our Legendre-SNN on a wider range of hyper-parameters in [10]. Note that Fang et al. [22] ($\curlyvee$) trained their SNN model with more than 0.5M learnable parameters, whereas our mv-LSNN has *no more than* 15K learnable parameters, therefore it struggles to outperform [22] on the *RacketSports* dataset.

E. mv-LSNN's Performance for Jamming Detection

Since *WirelessJamming* is our custom generated dataset, we build & evaluate our own *non-spiking* benchmark models on it, and then compare mv-LSNN against them. Our 4 benchmark models are: (a) Dense-ANN (composed of all-to-all connected `Dense` layers), (b): Echo State Network (ESN, composed of a static reservoir & a trainable readout layer), (c): ESN-Dense (composed of ESN & `Dense` layers), and (d): LSTM-Dense (composed of LSTM & `Dense` layers) – their architectural & hyper-parameter details are in the Appx I of Suppl doc. Note that all the benchmark models (except ESN) have comparable or more number of trainable parameters than the mv-LSNN corresponding to $d = 12$ & $K = 4$ for *WirelessJamming*; the ESN has significantly fewer parameters due to its conventionally simple design [55]. We mention the number of trainable parameters of mv-LSNN and benchmark models in Table III.

In Table II, for a *challenging* comparison of mv-LSNN, we present the *best* results of our 4 benchmark models; however, for a *fair* comparison, we compare mv-LSNN's accuracies with those of Dense-ANN, ESN, and ESN-Dense. We do *not* compare mv-LSNN with LSTM-Dense because unlike LSTM-Dense that has a *complex trainable* recurrence, mv-LSNN has a *simple non-trainable* recurrence; thus, it is expected that a similarly sized LSTM-based model will perform better. Considering the overall accuracy results in Table II, we see a clear trend - that as the E_b/N_0 of the jamming signal increases, the jamming detection performance of all the methods (generally) consistently improves – with a noticeable improvement between 0 dB & 1 dB E_b/N_0. Among Dense-ANN, ESN, and ESN-Dense models, we see that ESN performs the worst because of its minimalistic design with significantly fewer learnable parameters (as only its readout layer is trainable). Dense-ANN has more number of trainable parameters than the ESN, thus it performs better than ESN; however, since it does not have a recurrence component, it performs poorer than the hybrid ESN-Dense model that has a static recurrence followed by non-linear trainable `Dense` layers (- which enables ESN-Dense to not only capture temporal dynamics but also learn rich discriminatory features). The LSTM-Dense, as expected performs the best due to learnable recurrence and Dense layers.

TABLE II
ACCURACY (IN %) ON WIRELESSJAMMING DATASET FOR 6 E_b/N_0 LEVELS

Wireless-Jamming	mv-LSNN					Benchmark Models			
	slyr_best	*slyr_mean*	*l2sim_best*	*l2sim_mean*	*l2hw_best*	*Dense-ANN*	*ESN*	*ESN-Dense*	*LSTM-Dense*
E_b/N_0:*0dB*	**84.88**	**82.43**	**86.63**	**84.25**	**73.25**	64.63	58.50	70.88	93.63
E_b/N_0:*1dB*	**90.88**	**89.88**	**91.88**	**90.95**	**79.00**	68.50	61.38	77.88	97.75
E_b/N_0:*2dB*	**93.88**	**91.53**	**93.25**	**92.63**	**82.25**	72.63	67.25	81.75	98.63
E_b/N_0:*3dB*	**94.00**	**91.63**	**94.75**	**93.60**	**83.13**	74.88	68.75	82.75	98.13
E_b/N_0:*4dB*	**94.13**	**92.58**	**94.75**	**93.35**	**84.88**	76.63	66.50	84.13	98.13
E_b/N_0:*5dB*	**94.50**	**92.78**	**95.13**	**93.60**	**87.63**	77.88	67.75	86.88	98.88

Accuracies from mv-LSNN's deployment on CPU/GPU (i.e., $slyr_*$), on Loihi-2's simulation on CPU (i.e., $l2sim_*$), and on physical Loihi-2 (i.e., $l2hw_best$). For a fair comparison, we compare mv-LSNN's accuracies with those from ANN and ESN based models deployed on CPU/GPU. We do *not* compare with LSTM-Dense because of trainable recurrence in LSTM layer.

Note that the design of ESN-Dense model makes it best suited to compare with mv-LSNN because both the models have static recurrence followed by learnable `Dense` layers. As can be inferred from the Table II, the mv-LSNN - on all different backends and metrics - performs consistently better than ESN-Dense to detect jamming signals. We also see that mv-LSNN's results on CPU/GPU and on Loihi-2's *simulation* (i.e., $slyr_*$ and $l2sim_*$) are similar; however, on the *physical* Loihi-2 (i.e., $l2hw_best$), mv-LSNN performs poorly than on Loihi-2's *simulation* (i.e., $l2sim_best$). Ideally, the accuracies of a model on physical hardware and its simulation should match, however, this wasn't the case with mv-LSNN on *physical* Loihi-2 & its *simulation*. This wide mismatch is likely due to difference in affine transformation methods (in Lava) on *simulation* & *physical* Loihi-2 to meet the hardware constraints.

F. Demonstration Details on KPSC Loihi-2 Board

Here, we explain our video demonstration of jamming detection on Intel's KPSC Loihi-2 board (see Fig. 6). The SoM on Loihi-2 board is capable of running a Linux OS (Ubuntu 20.04 in our case) and is responsible for communication to-and-fro with the *physical* Loihi-2 chip (via the Loihi driver in NxCore); henceforth, we call this SoM (or the KPSC Loihi-2 board) as a Host. We connect this Host to a Router via an RJ45 Ethernet cable. Simultaneously, we connect our Dell Laptop (i7-5500U, 8GB) running Ubuntu 20.04 to the Router as well (via Ethernet); henceforth, we call this Laptop as the Super-host. In other words, a communication link between the Super-host (i.e., Laptop) and the Host (i.e., KPSC Loihi-2 board) is established via a Router; see Fig. 7 for the demonstration setup.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.



(a) Side 1 of KPSC Loihi-2 board (b) Side 2 of KPSC Loihi-2 board

Fig. 6. Intel's kapoho point single chip (KPSC) board; as name suggests, it has only one Loihi-2 chip. In (a) Blue wire: Ethernet, Red-Black wire: Power.

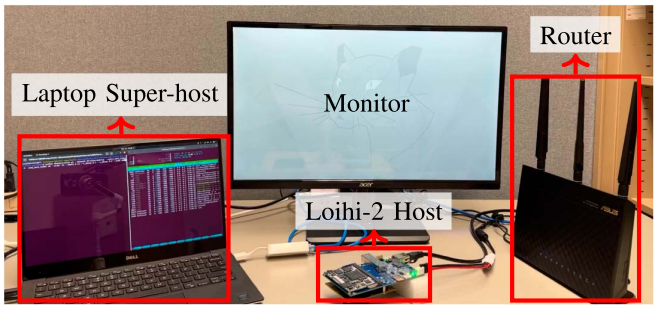


Fig. 7. Demonstration setup for jamming detection on KPSC Loihi-2 board. "laptop super-host" & "loihi-2 host" are connected via "router"; an external "monitor" is connected to "laptop super-host" to display progress (see video).

The Lava program to build the Loihi-2 deployable version of the (trained) mv-LSNN and the *WirelessJamming* dataset reside on the Super-host. Upon starting the execution of the demonstration on the Super-host, the wireless signals are sent from the Super-host to the Host, where the mv-LSNN executes on the Loihi-2 chip and infers on the received signal. The output spikes from the Loihi-2 chip are sent back to the Super-host where predicted classes are computed and are displayed in the demonstration (see video). We run our demonstration on a mix of 10 randomly chosen clean and noisy signals; note that a higher demonstration accuracy can be obtained by using a better trained mv-LSNN. Also note that we have intentionally slowed down our demonstration for the ease of viewing.

V. Discussion

We next present our discussion around mv-LSNN's overall performance, its energy-consumption on Loihi-2 (for jamming detection), and its advantages & limitations. Note that we compare mv-LSNN's energy-consumption (on Loihi-2) with that of our benchmark models on CPU (for jamming detection).

A. mv-LSNN's Overall Performance

SNNs are generally considered inferior to ANNs/RNNs with respect to performance accuracy. Considering mv-LSNN's best performance on 24 TSC datasets, we see that it outperforms the complex models such as LSTM-FCN & ESN-Dense (both on CPU/GPU) on 8 datasets when evaluated on *physical* Loihi-2 ($l2hw_best$ in Tables I & II) and on at least 11 datasets when deployed on CPU/GPU or Loihi-2's *simulation* ($slyr_best$ & $l2sim_best$ in Tables I & II). Given that the 24 datasets represent a variety of TSC domains, our mv-LSNN's performance presents a very promising outlook of SNNs (or Neuromorphic Computing) towards a wide range of TSC applications.

TABLE III
Energy Consumption and Number of Trainable Parameters

	mv-LSNN	Dense-ANN	ESN	ESN-Dense	LSTM-Dense
Energy (mJ):	16.87	4030.05	9126.34	10907.09	2037.87
Parameters:	14112	14738	101	14834	17714

Per-sample energy consumption of different models for jamming detection (on our *WirelessJamming* dataset), as well as their number of trainable parameters. Energy consumption of mv-LSNN is on the physical Loihi-2 chip, and that of rest of the models are on Intel Xeon W-2225 CPU.

B. Energy Consumption of mv-LSNN and Benchmark Models

Here we discuss the energy consumption of mv-LSNN on the *physical* Loihi-2 chip, and of the benchmark models on Intel Xeon W-2225 CPU. Note that all the energy metrics in milli-Joules (mJ) are obtained on our *WirelessJamming* dataset, and are presented in Table III. For uniformity of measuring energy consumption of different models on different backends, we set a hard-coded signal and compute energy consumption of only the forward pass through a model - such that it produces an output after receiving the input signal. With respect to measuring mv-LSNN's energy consumption on the physical Loihi-2 chip, we use our procedure mentioned in [14]. In short, for each *run* of profiling the mv-LSNN's energy consumption, we repeat the hard-coded signal for 500 duplicates, and compute the average energy-consumed for one signal (- say "energy$_{run_i}$"). We do so for 20 independent *runs* and then compute the mean of those 20 per-signal "energy$_{run_i}$" values; we refer this final averaged value as the "per-sample" energy consumption, which we report in Table III. With respect to measuring the energy consumption of our benchmark models on CPU, we follow the same aforementioned approach to obtain their "per-sample" energy consumption, and report in Table III. There, we see that our mv-LSNN on Loihi-2 is highly energy efficient, consuming just $16.87mJ$, whereas, Dense-ANN, ESN, ESN-Dense, & LSTM-Dense consume significantly high energy on CPU. With respect to the FPGA boards, [56], [57], & [58] report energy consumption of $38pJ$, $50pJ$, & $50pJ$ respectively (or even higher, e.g., $151pJ$ in [58]) - for per synaptic operation of their deployed SNN models ($pJ \Rightarrow$ pico-Joules). Note that Loihi-1 and Loihi-2 consume $23.6pJ$ [59] and $8.3pJ$ [60] respectively - for per synaptic operation. Thus, we can informatively expect mv-LSNN to consume more energy on typical FPGA boards than on the Loihi-2 chip.

C. Advantages and Limitations of mv-LSNN

Our mv-LSNN model clearly has a lot of advantages; they are - (1) *Simplicity*: mv-LSNN's architecture is very simple, it has a static recurrence that is easily implementable just by regular matrix operations (i.e., *no* neurons are involved), and the learnable component comprises only `Dense` layers, (2) *Hardware-amenability*: this simplicity of mv-LSNN makes

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

it highly amenable to custom ASIC or FPGA designs, (3) *High energy-efficiency*: mv-LSNN on Loihi-2 is highly energy-efficient than Dense-ANN (more than 200 x), ESN and ESN-Dense (more than 500 x), and LSTM-Dense (more than 100 x) - as can be seen in Table III. However, apart from these lucrative advantages of mv-LSNN, there is an obvious downside too. From the Tables I and II, it is apparent that mv-LSNN lags behind LSTM-based models. On one hand where the reservoir-based architecture attributes simplicity and hardware-amenability to mv-LSNN, the *lack* of complex and learnable recurrence (like in LSTM) renders mv-LSNN weaker compared to the LSTM-based models; thus, mv-LSNN performs poorly on more than 50% of the 24 datasets. Although, in another perspective, the positive performance of mv-LSNN on more than 40% of the 24 datasets does show its potential against LSTM-based models.

D. Ablation Study With Respect to Jamming Detection

With respect to jamming detection, although the mv-LSNN outperforms other "in-class" reservoir-based architectures (i.e., ESN and ESN-Dense), it fails to outperform the "out-of-class" LSTM-Dense. Note that, as mentioned in Sec III-B, the value of LDN's order d determines the richness of temporal features $\bar{x}[t]$, as well as the learning capacity of mv-LSNN. Therefore, we further experimented with large values of $d > 12$, and found that mv-LSNN's performance reaches quite close to that of LSTM-Dense; we present the details in Appx J of Suppl doc. However, when deploying such higher order mv-LSNN models on *physical* Loihi-2, we incurred memory errors - due to the LMT core reaching its peak memory use. Upon investigation, we found that for a 4-dim signal (e.g., *WirelessJamming*), the maximum order of LDNs in our **RES** (on LMT) can be 12 - before our implementation hits LMT core's memory limits.

VI. Conclusion

This work dove deeper into the technicalities of neuromorphic deployment of SNNs on Loihi-2 and their applications in 5G wireless domain. Herein, we implemented the multivariate version of Legendre-SNN [10] (i.e., mv-LSNN) *entirely* on Loihi-2. The mv-LSNN has a *non-spiking* reservoir followed by a *spiking* network; therefore, to avoid splitting it between CPU & Loihi-2, we presented our theoretical and technical specifics to implement the *non-spiking* component on Loihi-2's embedded LMT core (with the obvious deployment of *spiking* network on Neuro-Cores). Given the dearth of documentation to program LMT cores, our technical approach can be used by others to implement their SNNs *entirely* on Loihi-2, thereby capitalizing the most on Loihi-2 - as shown in Table III. Additionally, our mv-LSNN outperformed LSTM-based models on more than 40% of the 24 datasets (and close to 30% right on *physical* Loihi-2 chip) - this challenges the notion that SNNs are mostly inferior to conventional neural networks. To further strengthen our cause for leveraging neuromorphic computing for 5G wireless, we also presented a video demonstration of jamming detection on Intel's KPSC Loihi-2 board.

Coming to subsequent work directions, a number of avenues show up. One can work on improving the performance of mv-LSNN by training its static recurrent connections (i.e., LDN's state matrices), as well as by adding multiple spiking hidden layers to learn more abstract discriminatory features; the hidden layers are easily implementable on Loihi-2's Neuro-Cores. As demonstrated by the success of higher order LDN(s) - in case of jamming detection, one can use them for other datasets/applications too. Note that Loihi-2 has 6 LMT cores, therefore, to mitigate memory errors when implementing the reservoir (of higher order LDNs) on an LMT core, one can look into leveraging the other LMT cores as well. Also note that Loihi-2 supports reward-based on-chip learning, therefore, one can look into training mv-LSNN right on Loihi-2. Finally, apart from developing FPGA/ASIC designs for mv-LSNN, one can apply it for other 5G-&-Beyond wireless applications too.

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for modern deep learning research," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 9, 2020, pp. 13693–13696.

[2] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[3] R. Verdecchia, J. Sallou, and L. Cruz, "A systematic review of green AI," *Wiley Interdisciplinary Rev., Data Mining Knowl. Discovery*, vol. 13, no. 4, 2023, Art. no. e1507.

[4] Q. Liu and Z. Zhang, "Ultralow power always-on intelligent and connected SNN-based system for multimedia IoT-enabled applications," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15570–15577, Sep. 2022.

[5] H. Zheng and Y. Yi, "Spiking neural encoding schemes and STDP training algorithms for edge computing," in *Proc. 8th ACM/IEEE Symp. Edge Comput.*, 2023, pp. 365–371.

[6] G. Zhang et al., "Spiking neural networks in intelligent edge computing," *IEEE Consum. Electron. Mag.*, vol. 14, no. 4, pp. 66–75, Jul. 2024.

[7] Y. Liu, Z. Qin, and G. Y. Li, "Energy-efficient distributed spiking neural network for wireless edge intelligence," *IEEE Trans. Wireless Commun.*, vol. 23, no. 9, pp. 10683–10697, Sep. 2024.

[8] H. Zheng, J. Xu, L. Liu, and Y. Yi, "SNNOT: Spiking neural network with on-chip training for MIMO-OFDM symbol detection," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 4, pp. 1809–1823, Dec. 2024.

[9] L. Hu et al., "Enhanced wireless technology recognition method using energy-efficient spiking neural networks," in *Proc. 6th Int. Conf. Next Gener. Data-Driven Netw. (NGDN)*, Piscataway, NJ, USA: IEEE Press, 2024.

[10] R. Gaurav, T. C. Stewart, and Y. Yi, "Reservoir based spiking models for univariate time series classification," *Front. Comput. Neurosci.*, vol. 17, 2023, Art. no. 1148284.

[11] A. R. Voelker and C. Eliasmith, "Improving spiking dynamical networks: Accurate delays, higher-order synapses, and time cells," *Neural Comput.*, vol. 30, no. 3, pp. 569–609, 2018.

[12] A. Voelker, I. Kajić, and C. Eliasmith, "Legendre memory units: Continuous-time representation in recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32.

[13] S. H. Ahmed, J. Finkbeiner, and E. Neftci, "A hybrid SNN-ANN network for eventbased object detection with spatial and temporal attention," 2024, *arXiv:2403.10173*.

[14] R. Gaurav, T. C. Stewart, and Y. Yi, "Legendre-SNN on Loihi-2: Evaluation and insights," in *Proc. Workshop Mach. Learn. New Compute Paradigms (NeurIPS)*, 2024.

[15] R. Gaurav, S. Agarwal, T. C. Stewart, and Y. Yi, "Benchmarking deep legendre-SNN for time series classification – Analysis and enhancements," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 10, no. 1, pp. 858–869, Feb. 2026.

[16] F. Karim, S. Majumdar, H. Darabi, and S. Chen, "LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 6, pp. 1662–1669, 2018.

[17] F. Karim, S. Majumdar, H. Darabi, and S. Harford, "Multivariate LSTM-FCNS for time series classification," *Neural Netw.*, vol. 116, 2019.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.

[18] H. Akhlaghpasand, S. M. Razavizadeh, E. Björnson, and T. T. Do, "Jamming detection in massive MIMO systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 2, pp. 242–245, Apr. 2018.

[19] J. P. Dominguez-Morales et al., "Deep spiking neural network model for time-variant signals classification: A real-time speech recognition approach," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Rio De Janeiro, Brazil. Piscataway, NJ, USA: IEEE Press, 2018, pp. 1–8.

[20] A. Zhang, W. Zhu, and J. Li, "Spiking echo state convolutional neural network for robust time series classification," *IEEE Access*, vol. 7, pp. 4927–4935, 2019.

[21] A. Gautam and V. Singh, "CLR-based deep convolutional spiking neural network with validation based stopping for time series classification," *Appl. Intell.*, vol. 50, no. 3, pp. 830–848, 2020.

[22] H. Fang, A. Shrestha, and Q. Qiu, "Multivariate time series classification using spiking neural networks," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1–7.

[23] A. Lotfi Rezaabad and S. Vishwanath, "Long short-term memory spiking networks and their applications," in *Proc. Int. Conf. Neuromorphic Syst.,* 2020, pp. 1–9.

[24] Z. Yan, J. Zhou, and W.-F. Wong, "Energy efficient ECG classification with spiking neural network," *Biomed. Signal Process. Control*, vol. 63, 2021, Art. no. 102170.

[25] W. Ponghiran and K. Roy, "Hybrid analog-spiking long short-term memory for energy efficient computing on edge devices," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 581–586.

[26] G. Datta, H. Deng, R. Aviles, and P. A. Beerel, "Towards accurate, energy-efficient, & low-latency spiking LSTMs." 2022, *arXiv:2210.12613*.

[27] S. Dey, D. Banerjee, A. M. George, A. Mukherjee, and A. Pal, "Efficient time series classification using spiking reservoir," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 1–8.

[28] T. Ibad, S. J. Abdulkadir, N. Aziz, M. G. Ragab, and Q. Al-Tashi, "Hyperparameter optimization of evolving spiking neural network for time-series classification," *New Gener. Comput.*, vol. 40, no. 1, pp. 377–397, 2022.

[29] R. Gaurav, T. C. Stewart, and Y. C. Yi, "Spiking reservoir computing for temporal edge intelligence on Loihi," in *Proc. IEEE/ACM 7th Symp. Edge Comput. (SEC)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 526–530.

[30] Y. Feng, S. Geng, J. Chu, Z. Fu, and S. Hong, "Building and training a deep spiking neural network for ECG classification," *Biomed. Signal Process. Control*, vol. 77, 2022, Art. no. 103749.

[31] H. Peng et al., "Reservoir computing models based on spiking neural p systems for time series classification," *Neural Netw.*, vol. 169, pp. 274–281, 2024.

[32] X. Xiong et al., "Time series classification models based on nonlinear spiking neural P systems," *Eng. Appl. Artif. Intell.*, vol. 129, 2024, Art. no. 107603.

[33] W. Maass, T. Natschläger, and H. Markram, "Real-time computing without stable states: A new framework for neural computation based on perturbations," *Neural Comput.*, vol. 14, no. 11, pp. 2531–2560, 2002.

[34] H. Peng et al., "Nonlinear spiking neural P systems," *Int. J. Neural Syst.*, vol. 30, no. 10, 2020, Art. no. 2050008.

[35] S. Schliebs and N. Kasabov, "Evolving spiking neural network—A survey," *Evolving Syst.*, vol. 4, no. 2, pp. 87–98, 2013.

[36] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of continuous-valued deep networks to efficient event-driven networks for image classification," *Front. Neurosci.*, vol. 11, p. 682, 2017.

[37] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: Vgg and residual architectures," *Front. Neurosci.*, vol. 13, p. 95, 2019.

[38] F. Zenke and S. Ganguli, "SuperSpike: Supervised learning in multilayer spiking neural networks," *Neural Comput.*, vol. 30, no. 6, pp. 1514–1541, 2018.

[39] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019.

[40] L. Chiarello, P. Baracca, K. Upadhya, S. R. Khosravirad, and T. Wild, "Jamming detection with subcarrier blanking for 5G and beyond in industry 4.0 scenarios," in *Proc. IEEE 32nd Annu. Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 758–764.

[41] P. Kryszkiewicz and M. Hoffmann, "Open ran for detection of a jamming attack in a 5g network," in *Proc. IEEE 97th Veh. Technol. Conf. (VTC2023-Spring)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1–2.

[42] Y. Arjoune and S. Faruque, "Real-time machine learning based on hoeffding decision trees for jamming detection in 5G new radio," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 4988–4997.

[43] M. Hachimi, G. Kaddoum, G. Gagnon, and P. Illy, "Multi-stage jamming attacks detection using deep learning combined with kernelized support vector machine in 5G cloud radio access networks," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1–5.

[44] M. Varotto, S. Valentin, and S. Tomasin, "Detecting 5G signal jammers with autoencoders based on loose observations," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 160–165.

[45] M. Varotto, F. Heinrichs, T. Schürg, S. Tomasin, and S. Valentin, "Detecting 5G narrowband jammers with CNN, k-nearest neighbors, and support vector machines," in *Proc. IEEE Int. Workshop Inf. Forensics Secur. (WIFS)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 1–6.

[46] A. Hussain, N. Abughanam, J. Qadir, and A. Mohamed, "Jamming detection in IoT wireless networks: An edge-AI based approach," in *Proc. 12th Int. Conf. Internet Things,* 2022, pp. 57–64.

[47] Y. Wang, S. Jere, S. Banerjee, L. Liu, S. Shetty, and S. Dayekh, "Anonymous jamming detection in 5g with Bayesian network model based inference analysis," in *Proc. IEEE 23rd Int. Conf. High Perform. Switching Routing (HPSR)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 151–156.

[48] S. Jere, Y. Wang, I. Aryendu, S. Dayekh, and L. Liu, "Bayesian inference-assisted machine learning for near real-time jamming detection and classification in 5G new radio (NR)," *IEEE Trans. Wireless Commun.*, vol. 23, no. 7, pp. 7043–7059, Jul. 2024.

[49] J. Viana, H. Farkhari, P. Sebastiao, and V. P. G. Jimenez, "PCA-featured transformer for jamming detection in 5G UAV networks," 2024, *arXiv:2412.15312*.

[50] Y. Arjoune, F. Salahdine, M. S. Islam, E. Ghribi, and N. Kaabouch, "A novel jamming attacks detection approach based on machine learning for wireless communication," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 459–464.

[51] C. Eliasmith and C. H. Anderson, *Neural Engineering: Computation, Representation, and Dynamics in Neurobiological Systems.* Cambridge, MA, USA: MIT Press, 2003.

[52] D. P. Kingma, "Adam: A method for stochastic optimization," 2014, *arXiv:14126980*.

[53] J. Xu, L. Li, L. Zheng, and L. Liu, "Learning to estimate: A real-time online learning framework for MIMO-OFDM channel estimation," *IEEE Trans. Wireless Commun.*, vol. 24, no. 4, pp. 2634–2646, Apr. 2024.

[54] Z. Zhou, L. Liu, and H.-H. Chang, "Learning for detection: MIMO-OFDM symbol detection through downlink pilots," *IEEE Trans. Wireless Commun.*, vol. 19, no. 6, pp. 3712–3726, Jun. 2020.

[55] H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks-with an erratum note," Bonn, Germany: German National Research Center for Information Technology GMD Technical Report, 2001.

[56] R. Wang and A. van Schaik, "Breaking Liebig's law: An advanced multipurpose neuromorphic engine," *Front. Neurosci.*, vol. 12, p. 593, 2018, Art. no. 593.

[57] H. Irmak, F. Corradi, P. Detterer, N. Alachiotis, and D. Ziener, "A dynamic reconfigurable architecture for hybrid spiking and convolutional FPGA-based neural network designs," *J. Low Power Electron. Appl.*, vol. 11, no. 3, 2021, Art. no. 32.

[58] F. Corradi, G. Adriaans, and S. Stuijk, "Gyro: A digital spiking neural network architecture for multi-sensory data analytics," in *Proc. Drone Syst. Eng. Rapid Simul. Perform. Eval., Methods Tools,* 2021, pp. 9–15.

[59] M. Davies et al., "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan./Feb. 2018.

[60] A. H. Qahtani et al., "Neuromorphic edge artificial intelligence architecture for real-time surgical decision support: Integrating spiking neural networks with hybrid symbolic-neural reasoning," *J. Adv. Trends Med. Res.*, vol. 2, no. 2, pp. 288–294, 2025.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.



Ramashish Gaurav received the integrated dual degree from IIT-BHU (Varanasi), India, and the Masters of Applied Science degree from the University of Waterloo, Canada. He is currently working toward the Ph.D. degree with the BRain-Inspired Computing, Communication & Security (BRICCS) lab, Bradley Department of Electrical and Computer Engineering, Virginia Tech. He is a member of the Multifunctional Integrated Circuits and Systems (MICS) group with the Virginia Tech. His research interests include spiking neural networks, neuromorphic computing, AI & ML.



Sujata Sinha received the B.Tech. degree from Uttar Pradesh Technical University, India, and the M.S. degree in computer science from Auburn University. She is currently working toward the Ph.D. degree with the BRain-Inspired Computing, Communication & Security (BRICCS) lab, Bradley Department of Electrical and Computer Engineering, Virginia Tech. She is also part of a collaborative research effort with Argonne National Laboratory through the Data Reduction and Compression Thrust of the ZF Project. Her research interests include information theory, lossy compression, adversarial machine learning, and security of wireless communication systems.



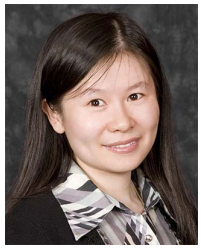
Chunxiao Lin received the Masters of Electrical and Computer Engineering degree from the University of Southern California (USC), US. He is currently working toward the Ph.D. degree with the BRain-Inspired Computing, Communication & Security (BRICCS) lab, Bradley Department of Electrical and Computer Engineering, Virginia Tech. He is also a member of the Multifunctional Integrated Circuits and Systems (MICS) group with the Virginia Tech. His research interests include reservoir computing, digital system design, AI & ML.



Terrence C. Stewart received the Ph.D. degree from Carleton University. He is currently a Research Officer with the National Research Council of Canada, as part of the Collaboration Centre, University of Waterloo, ON, Canada. His research interests include developing computational models of biological cognition, identifying the underlying algorithms, and evaluating whether those algorithms may have commercial application given modern energy-efficient neuromorphic hardware. He is also one of the developers of Nengo, a high-level library/design tool for developing large-scale computational neuroscience models, and running them efficiently on various hardware systems.



Lingjia Liu is a Professor and Bradley Senior Faculty Fellow with Bradley Department of Electrical Engineering and Computer Engineering, Virginia Tech (VT). He is also the Director of Wireless@VT. Prior to joining VT, he was an Associate Professor with EECS Department, University of Kansas (KU). He spent more than four years working in the Mitsubishi Electric Research Laboratory (MERL) and the Standards and Mobility Innovation Laboratory, Samsung Research America (SRA), where he received the Global Samsung Best Paper Award in 2008 and 2010. He was leading Samsung's efforts on multi-user MIMO, CoMP, and HetNets in 3GPP LTE/LTE-Advanced standards. His research interests include emerging technologies for 5G-Advanced / 6G cellular networks, including machine learning for wireless networks, massive MIMO, O-RAN, and mmWave communications.



Yang Yi (Senior Member, IEEE) is a Professor with Bradley Department of Electrical and Computer Engineering, Virginia Tech. Her research interests include Very large scale integrated (VLSI) circuits and systems, computer-aided design (CAD), neuromorphic computing and architectures for brain-inspired computing systems, and low-power circuits design with advanced nano-technologies for high-speed wireless systems. She leads the BRain-Inspired Computing, Communication & Security (BRICCS) lab, and is the Director of the Multifunctional Integrated Circuits and Systems (MICS) group with the Virginia Tech; she is also a member of the Wireless@VT group.

Authorized licensed use limited to: to IEEExplore provided by University Libraries | Virginia Tech. Downloaded on October 04,2026 at 17:05:58 UTC from IEEE Xplore. Restrictions apply.